

자기소개서

1. 나는 어떤 개발자인가

막히는 순간을 기록하고, 검증으로 증명하는 개발자입니다.

실무에서 쿼리 최적화나 DB Lock 같은 문제를 만나면, 1인 개발 환경에서 서비스 운영을 멈출 수 없었기에 당장 동작하는 방향으로 해결하고 넘어가는 경우가 많았습니다. “왜 이렇게 동작하는가”를 깊이 파볼 시간적 여유가 없었고, 그 부분이 계속 아쉬움으로 남았습니다.

그래서 공백기 동안 “What I Learned”라는 기술블로그를 운영하며, 업무에서 알게 쓰던 기술을 글로 재구성해 검증하는 습관을 들였습니다. Django ORM의 QuerySet 평가 시점, MVCC, 트랜잭션 격리 수준, `select_for_update`의 블록 동작까지 — 각 주제를 직접 실험하며 포스팅했습니다.

최근에는 LLM 프로젝트(LearnLog)에서 같은 원칙을 적용했습니다. LLM이 생성한 연습문제의 정답이 간헐적으로 틀리는 현상을 발견했을 때, 프롬프트 수정 후 “체감상 나아진 것 같다”에서 멈추지 않고, 동일 조건 비교 테스트와 자동 검증 체계를 구축해 실제로 오류가 줄었는지 수치로 확인한 뒤에야 릴리즈했습니다. 응답 시간이 68초까지 늘어났을 때도 마찬가지로, 작업별 벤치마크를 돌려 답변 생성과 경량 작업을 서로 다른 LLM으로 분리하고 48초로 단축된 것을 측정으로 확인했습니다.

경험으로 느낀 것을 기록으로 남기고, 개선했다고 생각한 것을 수치로 확인하는 과정을 반복하면서, 이게 저한테 맞는 개발 방식이라는 확신이 생겼습니다.

2. 가장 의미 있었던 기술적 경험

(1) 서비스 분리와 부하 분산

활성 고객 600+명, 월간 리밸런싱 시 약 18만~27만 건의 주문을 처리하는 주식 자동매매 서비스에서 리밸런싱 시점에 시스템 전체가 느려지는 현상이 반복됐습니다.

처음에는 쿼리 튜닝으로 해결하려 했지만, 리밸런싱이 아닌 시간대에는 문제가 없었기 때문에 쿼리 자체가 아니라 사내포털과 주문처리가 같은 프로세스·같은 DB를 공유하는 구조가 원인이라고 판단했습니다.

- 우선 Django 프로젝트 단위로 사내포털과 주문처리 서버를 분리했습니다. 그런데 리밸런싱 피크 시에는 단일 서버로 감당이 안 되는 문제가 남았습니다.
- 그래서 주문처리 서버에 **ELB + Auto Scaling**을 적용했습니다 (CPU 기준 최대 2대 자동 확장). 상시 고사양 인스턴스 대비 월 서버 비용이 약 400만 → 200~300만 원으로 줄었습니다.

3. 서버는 분리됐지만 DB는 여전히 하나였기 때문에, 주문처리 **DB**를 물리적으로 분리하고 사내포털에서는 Django DB Router로 두 DB를 동시 조회하도록 구성했습니다.

또한 APScheduler가 Gunicorn worker 4개마다 중복 실행되어 같은 작업이 4회씩 도는 문제를 발견해 스케줄러를 별도 프로젝트로 분리했고, 주문 DB가 8~17시만 active인 환경에서 스케줄러 스레드의 connection이 끊기지 않아 발생하던 lost connection 문제를 커넥션 명시적 close로 해결했습니다. → 상세 내용

(2) LLM 출력 검증 경험

LearnLog에는 저장된 학습 로그를 기반으로 LLM이 객관식 연습문제를 자동 생성하는 기능이 있습니다. 예를 들어 “Django ORM에서 N+1 문제란?”이라는 학습 로그가 있으면, 이를 바탕으로 4지선다 문제와 정답을 LLM이 만들어줍니다.

그런데 LLM이 생성한 정답 번호가 실제 정답 내용과 맞지 않는 현상이 간헐적으로 발생했습니다. 예를 들어 정답 내용은 3번 선지에 있는데, 정답 번호에는 1번이 들어가는 식입니다.

프롬프트를 수정하면 체감상 나아진 것 같았지만, LLM은 같은 입력에도 매번 다른 결과를 내기 때문에 “좋아진 것 같다”는 느낌만으로는 판단할 수 없었습니다. 그래서 수정 전후로 동일 조건 20회 반복 테스트를 돌리고, 각 결과를 별도 LLM이 자동 판정하는 검증 체계를 만들었습니다. 이를 통해 오류가 실제로 절반 수준으로 줄었음을 확인한 뒤 릴리즈했습니다.

별도 검증 코드와 테스트 과정은 [여기에서 확인하실 수 있습니다](#).

3. 경력 공백 기간의 활동과 성장

제 이력서에서 가장 먼저 눈에 띄는 것은 **3년 8개월의 공백**일 것입니다.

퇴사 결정의 배경. 첫 직장에서는 서비스 초기부터 안정화까지 경험하며 많이 배웠지만, 설계를 함께 검증해줄 동료가 부족한 환경이 길어지면서 **혼자 판단하는 방식의 한계**를 느꼈습니다. 더 깊이 성장하려면 환경의 전환이 필요하다고 판단해 퇴사를 선택했습니다.

공백기를 어떻게 채웠는가.

- **DB 심화 학습:** 실무에서 필요했지만 깊이 다루지 못했던 ORM 최적화, 트랜잭션, 동시성 제어 등을 **12편의 기술블로그 포스팅**으로 정리했습니다.
- **Docker 학습** (「**Docker 실무 입문, 컨테이너 빌드부터 배포까지**」 수강): Dockerfile 작성부터 Compose를 이용한 멀티 컨테이너 환경 구성까지, 컨테이너 기반 개발·배포 환경을 직접 설계할 수 있는 수준까지 학습했습니다. 이후 LearnLog에서 로컬과 배포 환경을 동일하게 유지하는 데 바로 활용했습니다.
- **LLM 프로젝트 LearnLog** (2026.01 첫 커밋 ~ 현재): 학습한 내용을 검색·정리·복습까지 한 곳에서 할 수 있는 시스템을 직접 만들었습니다. 기술 스택과 구현 상세는 이력서 프로젝트 항목에 정리되어 있습니다.

공백 전의 나와 지금의 나. 이전에는 문제를 해결해도 “동작하니까 됐다”에서 끝이었습니다. 원인을 정리하거나 다른 방법을 비교해보는 과정 없이 다음 업무로 넘어갔습니다. 지금은 문제를 만나면 원인을 글로 정리하고, 해결

방법을 바꿨을 때 실제로 나아졌는지 테스트로 확인하는 것이 자연스러운 습관이 되었습니다.

4. 함께 일하는 방식과 앞으로의 방향

이전 직장에서 가장 아쉬웠던 것은 **설계를 함께 검토할 동료**가 부족했다는 점이었습니다. 혼자 결정하고 혼자 검증 하다 보니, 더 나은 방법이 있었는지 확인할 기회가 없었습니다. 그래서 다음 회사에서는 PR 단위로 코드 리뷰가 이뤄지고, 설계 단계에서 “이 구조가 맞는지” 서로 의견을 주고받을 수 있는 환경을 중요하게 보고 있습니다.

한편, 연세대 LandFlow 프로젝트에서 비개발자인 기획자와 협업한 경험이 있습니다. Matlab GUI 개발 과정에서 요구사항을 받으면 개발 중인 화면 동작을 직접 보여주며 의도에 맞는지 수시로 확인했고, “이런 입력이 들어오면 어떻게 처리할지” 같은 예외 상황을 미리 정리해서 공유했습니다. 이 경험을 통해, 완성된 결과물을 한번에 전달하는 것보다 중간 과정을 자주 공유하면서 방향을 맞추는 것이 훨씬 효율적이라는 걸 느꼈습니다.

입사 후에는 먼저 서비스 도메인을 파악하는 데 집중하고, 이후에는 트러블슈팅이나 설계 판단을 기록으로 남기고, 개선은 수치로 확인하는 방식으로 팀에 기여하고 싶습니다.
